

Trabalho 1 - TBO

Relatório de Resultados

Ana Luisa Casotti, Clarice Curtinhas, Maria Julia Rigotti

Introdução

Este relatório irá abordar a execução do primeiro trabalho de Técnicas de Busca e Ordenação, que visa a resolução do problema de **agrupamento de espaçamento máximo**, por meio da implementação de uma MST (Árvore Geradora Mínima) e de algoritmos como **Union-Find** e **Kruskal**.

Como resultado da aplicação desses algoritmos, foi obtida uma complexidade de **$O(N^2 * \text{Log}N^2)$** e, ao fazermos a análise empírica, pudemos verificar quais as etapas mais custosas do programa.

Metodologia

Para a definição da implementação inicial do trabalho, o grupo se reuniu com o objetivo de planejar a estrutura geral da solução. Nessa etapa inicial, estabeleceu-se a criação de um arquivo principal (main), uma biblioteca “utils” para implementar funções usadas apenas na main e quatro Tipos Abstratos de Dados (TADs). A main seria responsável pelas funcionalidades básicas de leitura e escrita de arquivos, além de realizar as chamadas de funções necessárias para a execução dos algoritmos de processamento dos pontos.

Os TADs foram organizados da seguinte forma: um destinado à criação e manipulação de pontos e outro, de arestas; e os outros dois voltados à criação e gerenciamento de vetores (um de pontos e outro de arestas).

Para a implementação do union-find, definiu-se que a abordagem mais eficiente seria a utilização de **vetores** como estrutura de dados principal, permitindo acesso em tempo constante ($O(1)$) aos elementos por meio de seus índices. Essa escolha possibilita realizar buscas e uniões de forma eficiente, usando-se técnicas de otimização como o **Weighted Quick-Union com compressão de caminho**, tornando o processo de verificação de conectividade mais dinâmico.

Apesar da preparação e organização prévias do nosso código, as primeiras versões ainda se mostraram extremamente ineficientes. Durante nossos testes iniciais, conseguimos executar apenas os três primeiros casos de teste liberados pelo professor. Os demais casos estavam tão lentos que se tornaram inviáveis de executar. Dois gargalos de desempenho significativos foram encontrados: o vetor de alocação de pontos e o vetor de alocação de arestas. Para o vetor de alocação de pontos, utilizamos o Quick-Union com as otimizações mencionadas anteriormente para realizar o union-find dos pontos. Porém, dentro da

chamada de retorno de cluster/grupo, estávamos executando três loops distintos para determinar a posição de cada elemento em cada grupo. Algo análogo a isso ocorria na função de união das arestas, que chamava uma função de desalocação de arestas dentro de um loop, enquanto a função chamada também realizava loops para desalocar as arestas.

Além disso, em nossa implementação inicial, construímos o algoritmo de Kruskal utilizando um vetor para armazenar as arestas com seus respectivos pontos correspondentes e a distância entre eles. Após o armazenamento, utilizávamos o qsort do C para ordenar esse vetor com base na distância entre os pontos. Em seguida, verificávamos quais arestas eram válidas para gerar a MST e alocávamos outro espaço de memória para armazená-las em um vetor separado enquanto realizávamos as uniões de seus respectivos pontos. Repetíamos o processo de qsort para garantir que o vetor permanecesse ordenado. Depois disso, deletávamos essas uniões e as recriávamos após remover as k-1 maiores arestas válidas do vetor. Como pode ser observado, o código era extremamente ineficiente. Realizávamos alocações de vetores intermediários desnecessários e, embora o quick sort seja uma forma altamente otimizada de ordenar o vetor, seu uso frequente prejudicava significativamente o desempenho do programa.

Para resolver esses problemas, começamos eliminando os loops encadeados e substituindo-os por alternativas que utilizavam apenas loops indissociados. Por exemplo, usamos uma estrutura hash para obter as raízes dos grupos que seriam formados e alocar corretamente os espaços que cada grupo ocuparia, que poderiam ser encontrados posteriormente em tempo constante ($O(1)$). Apesar dessa técnica exigir várias passagens por vetor e a alocação de espaço para vetores grandes para criar esse sistema de hashes, o ganho de desempenho foi significativo. Após as alterações, conseguimos executar todos os testes liberados pelo professor, embora o último ainda levasse 17 minutos para ser concluído.

Na segunda etapa de alterações, removemos a desalocação de arestas dentro do loop da função de união de vetores. Essa segunda alteração teve um impacto notável, pois conseguimos executar o último caso em apenas 7 segundos.

Por fim, remodelamos nossa lógica de agrupamento de arestas usando uma estrutura de **heap**. Isso nos permitiu eliminar as passagens do vetor pelo quick sort e fornecer uma estrutura que permanecia facilmente ordenada para a função de união sem a necessidade de criar vetores secundários. Além disso, percebemos que seguir o passo a passo do algoritmo de Kruskal à risca prejudicava o nosso desempenho. Portanto, agrupamos a etapa de remoção de arestas inválidas com a etapa de remoção das k-1 maiores arestas válidas. Ao desconsiderarmos quaisquer arestas que não fossem utilizadas no final do algoritmo, conseguimos executar o quinto caso em nossos computadores pessoais em aproximadamente 1,3 segundos.

Análise de complexidade

A análise de complexidade será feita para cada TAD implementado e para a biblioteca *utils*.

1 - utils.c

int leArquivoEntrada(FILE *entrada, tVetorPontos *vetor_pontos): Realiza a leitura do arquivo de entrada. Assuma que:

N = número de linhas/pontos do arquivo;

M = número de coordenadas dos pontos no arquivo.

A função usa dois loops encadeados: o externo faz N iterações; o interno faz M iterações para cada iteração do loop externo. No loop externo os nomes dos N pontos são lidos (N tokens lidos). No loop interno são lidas M coordenadas. No interno também é feito o realloc de um vetor sempre que ele está cheio e precisa adicionar um novo elemento. No entanto, a capacidade do vetor é iniciada com duas posições e é dobrada sempre que realloc é chamado, ou seja, realloc é chamado poucas vezes. Nesse caso, realloc tem custo $O(1)$. Portanto, a complexidade é ditada pelas leituras do arquivo, que totalizam $N*(M+1)$.

Complexidade final da função: $O(NM)$.

2 - ponto.c

tPonto* criaPonto(char *id, double *coordenadas, int qtdCoord): Aloca um ponteiro do tipo *tPonto* e inicializa seus campos. Um desses campos é um vetor que é alocado para guardar um número M de coordenadas. A função realiza um loop para preencher o vetor, o qual realiza M iterações. A cada iteração, são feitos dois acessos a vetores: um de leitura (do vetor passado como parâmetro) e um de escrita (no vetor da estrutura). Portanto, a função faz $2*M$ acessos à memória.

Complexidade final da função: $O(M)$.

double distanciaEuclidiana(tPonto *p1, tPonto *p2): Calcula a distância euclidiana entre dois pontos. A função só realiza um loop, o qual faz M iterações (M = número de coordenadas dos pontos). A cada iteração, são feitos quatro acessos à memória e algumas operações aritméticas. Tanto o acesso quanto as operações são de tempo constante.

Complexidade final da função: $O(M)$.

3 - vetor_pontos.c

static int comparaPontos(const void *a, const void *b): Compara os pontos usando o nome/identificador como chave. Assuma N = tamanho da menor string. A função usa *strcmp*, que faz N comparações no pior caso.

Complexidade final da função: $O(N)$.

static int comparaGrupos(const void *a, const void *b): Compara os grupos/clusters usando o nome/identificador do primeiro ponto de cada grupo como chave. Assuma N = tamanho da menor string. A função usa *strcmp*, que faz N comparações no pior caso.

Complexidade final da função: $O(N)$.

tVetorPontos* criaVetorPontos(): Realiza somente operações primitivas em C. Inicializa as variáveis e aloca ponteiro para a *struct tVetorPontos*, que são operações de tempo constante $O(1)$; Aloca um array de uma dimensão, que aloca espaço para todos os pontos que serão armazenados, complexidade $O(N)$.

Complexidade final dessa função: $O(N)$.

void adicionaPonto(tVetorPontos *vetor, tPonto *p): Adiciona um ponto no vetor de pontos. A cada chamada dessa função, dois vetores são realocados caso estejam cheios. Seja N = tamanho do vetor ao entrar na função. O custo de cada realloc é de N cópias, portanto, $2*N$ é o número total de cópias feitas pela função no pior caso.

Complexidade final dessa função: $O(N)$.

void geraArestas(tVetorPontos *vetorPontos, tVetorArestas *vetorArestas): Realiza o cálculo de todas as combinações de distâncias entre os pares de pontos do vetor. Para isso, percorre dois for, o que resulta na complexidade $(N(N - 1))/2 \sim N^2$. Chama as funções *inicializaAresta* e *adicionaAresta*, que são $O(1)$, e a função *distanciaEuclidiana*, que é $O(M)$.

Complexidade final dessa função: $O(N^2M)$.

int findPontos(tVetorPontos *vetor, int ind): O algoritmo do Union-Find é utilizado para buscar a raiz de um elemento do vetor, aplicando a técnica de weighted com compressão de caminho para diminuir o tamanho da árvore enquanto faz o percurso. Como a árvore final tem altura muito pequena, a complexidade dessa técnica é quase constante e dada por $O(N + M \log^* N)$, com M sendo o número de chamadas das estruturas do Union-Find.

Complexidade final dessa função: $O(\log N)$.

int connectedPontos(tVetorPontos vetor, int ind1, int ind2): Realiza duas chamadas da função *findPontos*, que tem complexidade $O(\log N)$.

Complexidade final dessa função: $O(\log N)$.

void unionPontos(tVetorPontos *vetor, int ind1, ind2): Realiza a união entre dois conjuntos no algoritmo Union-Find. Inicialmente, são feitas duas chamadas da função *findPontos*. Em seguida, é utilizada a técnica de Weighted Quick-Union, conectando a árvore de menor tamanho à árvore de maior tamanho, com operações de custo $O(1)$. Portanto, o custo final é resultado dos dois finds.

Complexidade final dessa função: $O(\log N)$.

tGrupo* retornaGrupo(tVetorPontos *vetor, int n): Realiza dois loops para descobrir as raízes e os pontos que pertencem ao mesmo grupo, percorrendo o vetor de n elementos e chamando a função *findPontos*, o que gera um custo de $O(2N\log N)$. Para a ordenação interna dos grupos, o quick sort tem um custo de $O(N\log N)$, enquanto a ordenação entre os grupos apresenta custo $O(k\log k)$, com k sendo o número de grupos. Complexidade final dessa função: $O(N\log N)$.

void imprimeGrupos(tVetorPontos *vetor, int n, FILE *saida): Começa chamando a função *retornaGrupo*, que tem custo dominante de $O(N\log N)$. Depois, chama a função *imprimePonto* para cada ponto de todos os grupos, gerando um custo de $O(N)$.

Complexidade final dessa função: $O(N\log N)$.

void desalocaGrupos(tGrupo *grupos, int k): Percorre os k grupos e libera os pontos pertencentes a cada grupo.

Complexidade final dessa função: $O(k)$.

void desalocaVetorPontos(tVetorPontos *vetor): Realiza um loop por todo o vetor de pontos, chamando a função *desalocaPonto* em todas as iterações. Como o vetor de coordenadas do ponto tem tamanho M , a complexidade é dada por $O(NM)$. Os frees realizados para os elementos alocados são $O(1)$.

Complexidade final da função: $O(NM)$.

4 - aresta.c

Não consideramos relevante trazer a análise das funções desse TAD, pois foi verificado que todas têm complexidade $O(1)$.

5 - vetor_arestas.c

Valor de $E = (N * (N - 1)) / 2 = \sim N^2$

tVetorArestas* criaVetorArestas(int numPontos): Realiza somente operações primitivas em C. Inicializa as variáveis e aloca um ponteiro para a *struct tVetorArestas*, que são operações de tempo constante $O(1)$; Aloca um array de uma dimensão, que aloca espaço para todos os pontos que serão armazenados, complexidade $O(E)$.

Complexidade final dessa função: $O(E)$.

void adicionaAresta(tVetorArestas *vetor, tAresta *aresta): Realiza uma atribuição de variável a um índice do vetor e incrementa o tamanho do mesmo, realizado em tempo constante.

Complexidade final da função: $O(1)$.

void constroiHeap(tVetorArestas *vetor): Realiza $\sim E/2$ iterações no loop, em todas as iterações chama a função *heapifyDown* que tem complexidade $O(\text{Log}E)$, logo complexidade total = $(E * \text{Log}E)/2$.

Complexidade final da função: $O(E * \text{Log}E)$.

void heapifyDown(tVetorArestas *vetor, int i): Funciona como uma árvore binária, utiliza os cálculos de filhos da heap para caminhar pelo vetor em no máximo $\text{Log}E$, realizando as trocas de ponteiro ($O(1)$) para andar com o elemento pelo vetor.

Complexidade final da função: $O(\text{Log}E)$.

tAresta *removeMin(tVetorArestas *vetor): Realiza atribuições de variáveis, em tempo constante, chama ao final a função *heapifyDown* que tem complexidade $O(\text{Log}E)$.

Complexidade final da função: $O(\text{Log}E)$.

void unionArestas(tVetorArestas *vetorArestas, tVetorPontos *vetorPontos, int tam): Realiza atribuições de variáveis, em tempo constante; Faz um loop que valida os vetores e termina quando o número de uniões que devem ser feitas termina, máximo de iterações feitas por essa função $E-k$ (consideramos pior caso onde o if será falso a maioria das vezes); Função *connectedPontos* chamada em todas as iterações, $((E-k) * O(\text{Log}N))$; Função *unionPontos* chamada apenas $N-k$ vezes, $((N-k) * O(\text{Log}N))$.

Complexidade final da função: $O(E\text{Log}N)$.

void desalocaVetorArestas(tVetorArestas *vetor): Realiza um loop por todo o vetor de arestas chamando a função *desalocaAresta* em todas as iterações, complexidade $O(1)$; Feitos os frees no vetor de arestas e na struct vetor, free é feito em tempo constante.

Complexidade final da função: $O(E)$.

Complexidade final do trabalho: O custo total é dominado pelas funções mais custosas. São elas:

- *geraArestas*: $O(N^2M)$;
- *constroiHeap*: $O(E * \text{Log}E) = O(N^2 * \text{Log}N^2)$;
- *unionArestas*: $O(E\text{Log}N) = O(N^2\text{Log}N)$;
- *retornaGrupo*: $O(N\text{Log}N)$.

Como essas quatro funções são as mais custosas para o código, a complexidade final é de $O(N^2M + N^2 * \text{Log}N^2 + N^2 * \text{Log}N + N\text{Log}N) \sim O(N^2 * \text{Log}N^2)$.

Análise empírica

Análise do teste 4:

- 2500 pontos;
- 5 grupos;
- 5 coordenadas por ponto.

Tempo total: 0,351s

Etapas	% do Tempo Total
Leitura	1,56
Cálculo das distâncias	32,22
Ordenação das distâncias + criação da MST	66,96
Identificação dos grupos	0,43
Escrita	0,03

Análise do teste 5:

- 5000 pontos;
- 10 grupos;
- 10 coordenadas por ponto.

Tempo total: 1,357s

Etapas	% do Tempo Total
Leitura	1,18
Cálculo das distâncias	38,43
Ordenação das distâncias + criação da MST	59,45
Identificação dos grupos	0,22
Escrita	0,02

Conclusão:

Ao observar as tabelas, fica claro que a etapa mais custosa foi a de ordenação das distâncias e criação da MST. Esse resultado condiz com a complexidade das funções que

realizam essa etapa, que são as funções ***constroiHeap***, com custo $O(N^2 * \text{Log}N^2)$, e ***unionArestas***, com custo $O(N^2 \text{Log}N)$, onde N é o número de pontos. Essas são as duas funções mais custosas do programa.

Logo em seguida, vem a etapa de cálculo das distâncias, realizada pela função ***geraArestas***, que tem custo $O(N^2M)$. Como nos testes rodados $M < \text{Log}N$, faz sentido que essa etapa fique em segundo lugar em termos de maior tempo de execução.

Portanto, podemos afirmar que as porcentagens calculadas estão de acordo com a análise de complexidade feita anteriormente.